# Diabetes Risk Classification

## Machine Learning Analysis Report

*BRFSS 2015 Health Indicators Dataset*

**Models Evaluated:**

Softmax Regression • K-Nearest Neighbours • Neural Network

*2026*

*Ziad Islam Hosny - 22010778*

*Mahmoud Magdy - 2201195*

*Jana Mohamed - 22010691*

# 1. Introduction

Diabetes is a chronic metabolic disease affecting hundreds of millions of people worldwide. Early and accurate classification into **No Diabetes**, **Prediabetes**, and **Diabetes** categories is clinically valuable for preventive intervention. This report presents a complete machine-learning pipeline applied to the publicly available **BRFSS 2015 Health Indicators** dataset, encompassing data preparation, feature selection, class-imbalance handling, model training and hyperparameter tuning, and rigorous evaluation across three model families.

The three modelling approaches compared are: (1) **Softmax (Multinomial Logistic) Regression** — a linear baseline; (2) **K-Nearest Neighbours (KNN)** — a non-parametric, distance-based model; and (3) a **Feed-Forward Neural Network** — a multi-layer architecture capable of learning non-linear feature interactions.

# 2. Dataset Overview

**Source:** diabetes_012_health_indicators_BRFSS2015.csv

**Target variable:** Diabetes_012 — three ordinal classes (0 = No Diabetes, 1 = Prediabetes, 2 = Diabetes)

**Features:** 21 health indicators including BMI, blood pressure, cholesterol, physical activity, smoking status, general health rating, age, education, and income.

## 2.1 Class Imbalance

The dataset is strongly imbalanced. The **No Diabetes** class constitutes roughly 84% of records, **Diabetes** approximately 14%, and **Prediabetes** fewer than 2%. This severe skew is the central challenge of the project: naive models that ignore imbalance will achieve high accuracy by predicting the majority class almost exclusively, while failing on the clinically important minority classes.

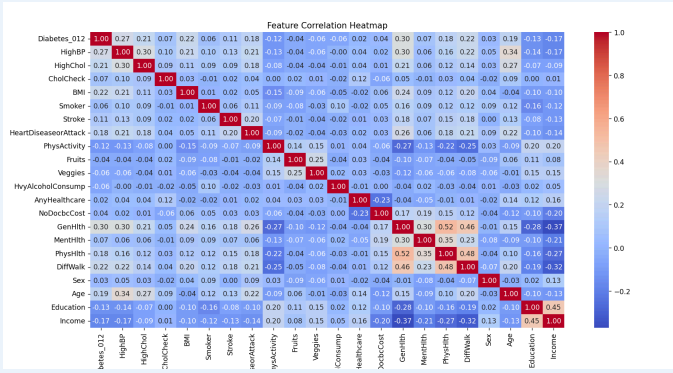



*Figure 1 — Feature Correlation Heatmap (dataset/correlationMatrix.py)*

**File:** `dataset/correlationMatrix.png`

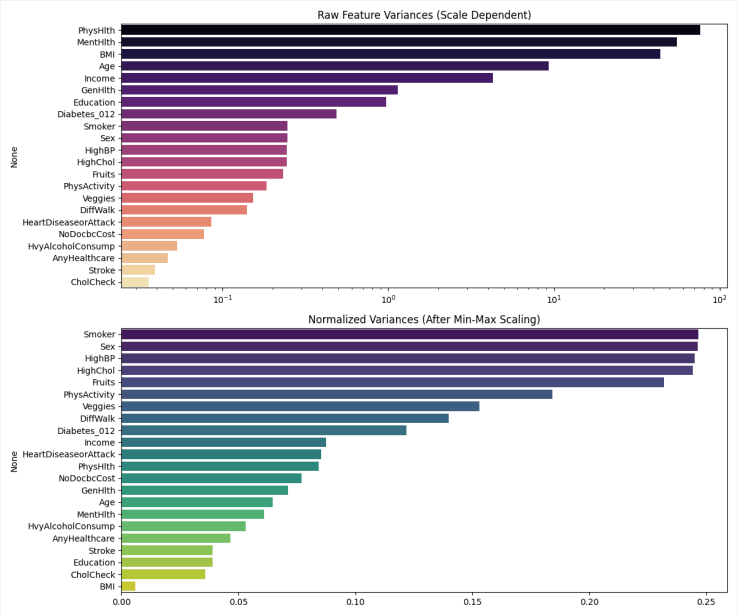



*Figure 2 — Raw and Normalised Feature Variances (dataset/variances.py)*

**File:** `dataset/variances_comparison.png`

# 3. Data Preparation

## 3.1 Train / Validation / Test Split

The dataset was partitioned with stratified sampling to preserve class proportions:

- 70 % Training set
- 10 % Validation set (used for hyperparameter tuning)
- 20 % Test set (held out for final, unbiased evaluation)

A fixed random seed (42) ensures reproducibility across all experiments.

## 3.2 Feature Scaling

All features were standardised using **StandardScaler** (zero mean, unit variance). The scaler was fitted exclusively on training data and applied to the validation and test sets to prevent data leakage.

### 3.3 Feature Selection

Automatic feature selection was performed using **SelectKBest** with the ANOVA F-statistic (`f_classif`). Four values of *k* were evaluated: 15, 18, 20, and 21. This allows the model to operate on only the most statistically discriminating features, reducing noise and overfitting risk.

### 3.4 Handling Class Imbalance

Three resampling strategies were systematically compared against a no-resampling baseline:

- Baseline (No resampling) — trains on the original skewed distribution.
- Random Oversampling — duplicates minority-class samples until all classes are balanced.
- Random Undersampling — removes majority-class samples to match the minority.
- SMOTE (Synthetic Minority Over-sampling Technique) — generates new synthetic minority samples by interpolating between existing ones, reducing the risk of simple duplication overfitting.

SMOTE is the theoretically preferred technique because it enriches the minority class with **novel synthetic examples** rather than exact duplicates, helping the classifier learn more general decision boundaries for Prediabetes and Diabetes.

## 4. Model Architectures & Hyperparameter Tuning

### 4.1 Softmax (Multinomial Logistic) Regression

Softmax regression is a generalisation of logistic regression to multiple classes. It fits a linear decision boundary in feature space, learning a weight vector per class. The regularisation strength **C** controls the trade-off between fitting the training data and penalising large weights.

**Hyperparameter search:** $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$, selection criterion: validation Macro-F1.

**Solver:** L-BFGS with max_iter = 1 000.

### 4.2 K-Nearest Neighbours (KNN)

KNN classifies a point by majority vote among its **k** nearest neighbours in the scaled feature space. It is non-parametric and makes no assumptions about the underlying distribution.

**Hyperparameter search:** $k \in \{3, 5, 11, 21\}$, metric $\in$ {Euclidean, Manhattan}, selection criterion: validation Macro-F1.

### 4.3 Feed-Forward Neural Network

The neural network architecture consists of four fully-connected layers:

- Input → 128 neurons (BatchNorm, ReLU, Dropout 0.3)
- 128 → 64 neurons (BatchNorm, ReLU, Dropout 0.2)

- 64 → 32 neurons (ReLU)
- 32 → 3 output neurons (raw logits)

**Optimiser:** AdamW | **Learning rate:** 0.0005 | **Batch size:** 64 | **Epochs:** 40

**Loss function:** Cross-Entropy with class weights computed by `compute_class_weight('balanced')`, giving higher loss to misclassifications of minority classes even when the training data has been resampled.

**Model selection:** The checkpoint with the highest validation Macro-F1 is saved to `models1/<config>/` and reloaded for final test-set evaluation.

**Hyperparameter search:** All combinations of k ∈ {15,18,20,21} and imbalance method ∈ {None, oversampling, undersampling, SMOTE} were trained, yielding 16 neural-network variants.

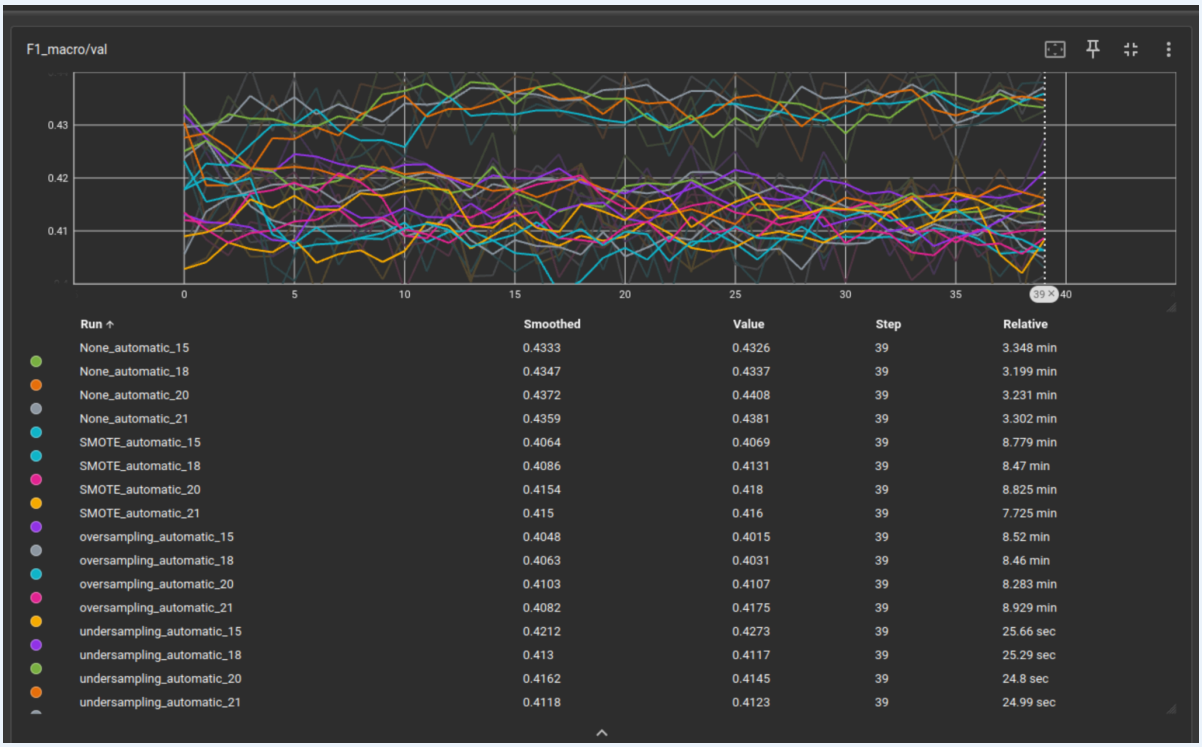
### 4.3.1 TensorBoard Experiment Tracking

Training runs are fully instrumented with **TensorBoard** via PyTorch's `SummaryWriter`. A separate log directory is created per configuration under `runs/<imbalance>_<selection>_<k>/`, enabling side-by-side comparison of all 16 variants in a single TensorBoard session.

The following scalars are logged throughout training:

- Loss/train_step — batch-level training loss at every optimiser step, providing a fine-grained view of convergence within each epoch.
- Loss/train_epoch — cumulative training loss summed over all batches in the epoch.
- Accuracy/val — validation accuracy recorded after each epoch.
- F1_macro/val — validation Macro-F1 after each epoch; this is the primary model-selection criterion.
- Best/F1_macro — logged each time a new best checkpoint is saved, making it easy to track when peak performance was reached across epochs.
- Test/accuracy, Test/macro_f1, Test/weighted_f1 — final test-set metrics logged once per configuration for direct cross-run comparison in the TensorBoard scalars panel.

To inspect all runs, execute `tensorboard --logdir runs/` from the project root. This provides learning-curve visualisations for every configuration, making it straightforward to detect underfitting, overfitting, or unstable training dynamics across the 16 variants.




| Run | Smoothed | Value | Step | Relative |
|---|---|---|---|---|
| None_automatic_15 | 0.4333 | 0.4326 | 39 | 3.348 min |
| None_automatic_18 | 0.4347 | 0.4337 | 39 | 3.199 min |
| None_automatic_20 | 0.4372 | 0.4408 | 39 | 3.231 min |
| None_automatic_21 | 0.4359 | 0.4381 | 39 | 3.302 min |
| SMOTE_automatic_15 | 0.4064 | 0.4069 | 39 | 8.779 min |
| SMOTE_automatic_18 | 0.4086 | 0.4131 | 39 | 8.47 min |
| SMOTE_automatic_20 | 0.4154 | 0.418 | 39 | 8.825 min |
| SMOTE_automatic_21 | 0.415 | 0.416 | 39 | 7.725 min |
| oversampling_automatic_15 | 0.4048 | 0.4015 | 39 | 8.52 min |
| oversampling_automatic_18 | 0.4063 | 0.4031 | 39 | 8.46 min |
| oversampling_automatic_20 | 0.4103 | 0.4107 | 39 | 8.283 min |
| oversampling_automatic_21 | 0.4082 | 0.4175 | 39 | 8.929 min |
| undersampling_automatic_15 | 0.4212 | 0.4273 | 39 | 25.66 sec |
| undersampling_automatic_18 | 0.413 | 0.4117 | 39 | 25.29 sec |
| undersampling_automatic_20 | 0.4162 | 0.4145 | 39 | 24.8 sec |
| undersampling_automatic_21 | 0.4118 | 0.4123 | 39 | 24.99 sec |

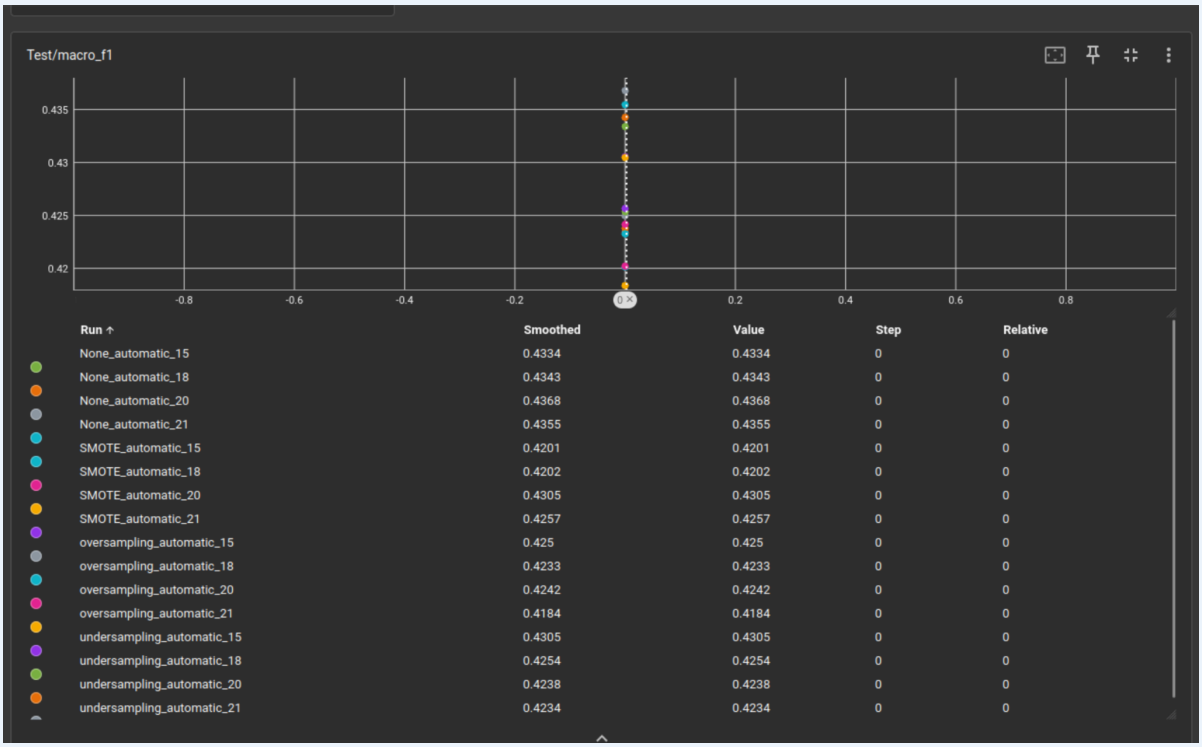
*Figure 13 — TensorBoard: F1_macro/val curves for all 16 Neural Network configurations*

**File:** `screenshots/tensorboard_f1_macro_val.png` `[screenshot from TensorBoard UI]`

| Run | Smoothed | Value | Step | Relative |
|---|---|---|---|---|
| None_automatic_15 | 0.9447 | 0.8378 | 110,999 | 3.419 min |
| None_automatic_18 | 0.9921 | 1.0104 | 110,999 | 3.266 min |
| None_automatic_20 | 0.9099 | 0.819 | 110,999 | 3.303 min |
| None_automatic_21 | 0.8331 | 0.7868 | 110,999 | 3.389 min |
| SMOTE_automatic_15 | 0.8371 | 0.6865 | 280,519 | 8.986 min |
| SMOTE_automatic_18 | 0.809 | 0.7488 | 280,519 | 8.685 min |
| SMOTE_automatic_20 | 0.923 | 1.046 | 280,519 | 9.034 min |
| SMOTE_automatic_21 | 0.7434 | 0.7309 | 280,519 | 7.907 min |
| oversampling_automatic_15 | 0.7169 | 0.5002 | 280,519 | 8.676 min |
| oversampling_automatic_18 | 1.0548 | 1.1763 | 280,519 | 8.633 min |
| oversampling_automatic_20 | 0.8721 | 0.8056 | 280,519 | 8.467 min |
| oversampling_automatic_21 | 0.8918 | 0.977 | 280,519 | 9.18 min |
| undersampling_automatic_15 | 0.9752 | 1.0822 | 6,079 | 25.88 sec |
| undersampling_automatic_18 | 0.8826 | 0.8486 | 6,079 | 25.56 sec |
| undersampling_automatic_20 | 0.9026 | 0.9139 | 6,079 | 25.08 sec |
| undersampling_automatic_21 | 0.8863 | 0.8533 | 6,079 | 25.3 sec |

*Figure 14 — TensorBoard: Loss/train_epoch curves for all 16 Neural Network configurations*

**File:** `screenshots/tensorboard_loss_train_epoch.png` `[screenshot from TensorBoard UI]`




| Run | Smoothed | Value | Step | Relative |
|---|---|---|---|---|
| None_automatic_15 | 0.4334 | 0.4334 | 0 | 0 |
| None_automatic_18 | 0.4343 | 0.4343 | 0 | 0 |
| None_automatic_20 | 0.4368 | 0.4368 | 0 | 0 |
| None_automatic_21 | 0.4355 | 0.4355 | 0 | 0 |
| SMOTE_automatic_15 | 0.4201 | 0.4201 | 0 | 0 |
| SMOTE_automatic_18 | 0.4202 | 0.4202 | 0 | 0 |
| SMOTE_automatic_20 | 0.4305 | 0.4305 | 0 | 0 |
| SMOTE_automatic_21 | 0.4257 | 0.4257 | 0 | 0 |
| oversampling_automatic_15 | 0.425 | 0.425 | 0 | 0 |
| oversampling_automatic_18 | 0.4233 | 0.4233 | 0 | 0 |
| oversampling_automatic_20 | 0.4242 | 0.4242 | 0 | 0 |
| oversampling_automatic_21 | 0.4184 | 0.4184 | 0 | 0 |
| undersampling_automatic_15 | 0.4305 | 0.4305 | 0 | 0 |
| undersampling_automatic_18 | 0.4254 | 0.4254 | 0 | 0 |
| undersampling_automatic_20 | 0.4238 | 0.4238 | 0 | 0 |
| undersampling_automatic_21 | 0.4234 | 0.4234 | 0 | 0 |

*Figure 15 — TensorBoard: Test/macro_f1 scalar comparison across all configurations*

**File:** `screenshots/tensorboard_test_macro_f1.png` `[screenshot from TensorBoard scalars panel]`

## 5. Choice of Evaluation Metric

**Which metric matters most for this dataset?** Given the severe three-class imbalance (≈84 / 2 / 14 %), **Macro-F1** is the most meaningful and reliable metric for this task.

To understand why, consider the four available metrics:

- Accuracy / Micro-F1 — these are equivalent in multi-class settings and are dominated by the majority class. A model that always predicts 'No Diabetes' achieves ~84% accuracy while being completely useless clinically. High accuracy therefore provides a misleading picture of true model quality.
- Weighted-F1 — weights each class's F1 by its support (sample count), so the majority class still dominates the average. Better than plain accuracy, but still biased toward the majority.

- Macro-F1 — computes F1 independently for each class and averages them with equal weight. This means that even though Prediabetes represents only ~2% of samples, its F1-score contributes equally to the overall score. A model that entirely ignores Prediabetes will suffer a large Macro-F1 penalty regardless of how well it handles 'No Diabetes'. This makes Macro-F1 the most honest indicator of balanced performance across all three clinical classes.

**Clinical justification:** Missing a Prediabetes or Diabetes diagnosis carries significantly higher patient risk than a false positive. A metric that gives equal weight to all classes aligns better with clinical priorities than one dominated by the easy majority class.

# 6. Results Summary

Table 1 presents the test-set metrics for all trained models, sorted by Macro-F1 (the primary ranking criterion).

***Table 1 — Full Model Performance Comparison (sorted by Macro-F1)***

| Model | Imbalance | k | Accuracy | F1-Micro | F1-Macro | F1-Weighted |
|---|---|---|---|---|---|---|
| Neural Network | None | 21 | 0.7044 | 0.7044 | 0.4404 | 0.7515 |
| Neural Network | None | 18 | 0.7007 | 0.7007 | 0.4382 | 0.7473 |
| Neural Network | None | 20 | 0.6969 | 0.6969 | 0.4375 | 0.7460 |
| Neural Network | Undersampling | 21 | 0.6804 | 0.6804 | 0.4339 | 0.7408 |
| Neural Network | None | 15 | 0.6984 | 0.6984 | 0.4333 | 0.7455 |
| Neural Network | SMOTE | 20 | 0.6798 | 0.6798 | 0.4318 | 0.7405 |
| Neural Network | Oversampling | 21 | 0.6711 | 0.6711 | 0.4284 | 0.7329 |
| Neural Network | SMOTE | 15 | 0.6401 | 0.6401 | 0.4169 | 0.7160 |
| KNN | Baseline | — | 0.8320 | 0.8320 | 0.3973 | 0.8031 |
| KNN | SMOTE | — | 0.7095 | 0.7095 | 0.4107 | 0.7475 |
| Softmax Reg. | Baseline | — | 0.8456 | 0.8456 | 0.3903 | 0.8065 |
| Softmax Reg. | SMOTE | — | 0.6406 | 0.6406 | 0.4222 | 0.7180 |

# 7. Model Comparison & Analysis

## 7.1 Which Model Performed Best?

The **Neural Network with no resampling and k = 21 features** achieved the highest Macro-F1 of **0.4404**, followed closely by Neural Network (None, k=18) at 0.4382 and Neural Network (None, k=20) at 0.4375. This makes the no-resampling neural network the best overall model when measured by Macro-F1.

In terms of raw accuracy, the **Softmax Regression Baseline** scored highest (0.8456), followed by KNN Baseline (0.8320). However, as explained in Section 5, these high accuracy figures are misleading — both models achieve them largely by predicting the dominant 'No Diabetes' class, as their correspondingly low Macro-F1 scores (0.3903 and 0.3973 respectively) reveal.

### 7.1.1 Softmax Regression (Linear Model)

The softmax baseline is a simple, interpretable model. With no resampling it achieved 84.56% accuracy but only 0.3903 Macro-F1 — the lowest Macro-F1 of all tested models. This indicates it heavily favours the majority class. Its decision boundaries are linear and cannot capture the complex non-linear interactions between health indicators that differentiate Prediabetes from the other classes. With SMOTE applied, Macro-F1 improved to 0.4222 while accuracy dropped sharply to 64.06%, illustrating the accuracy/minority-class recall trade-off inherent in resampling.

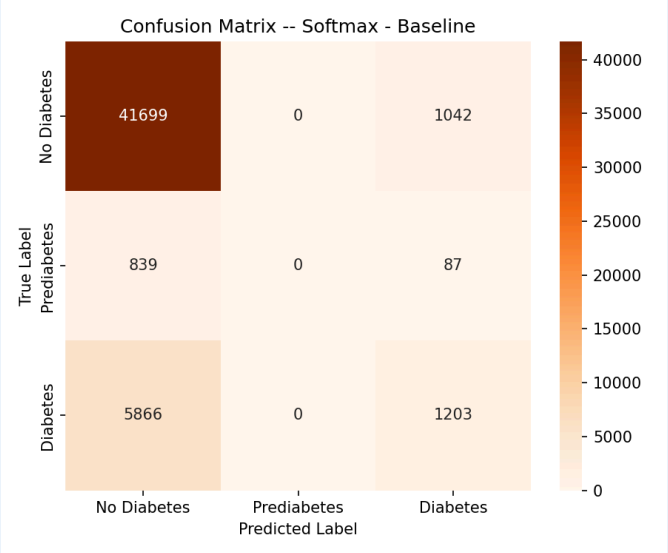



*Figure 3 — Softmax Regression: Confusion Matrix (Baseline)*

**File:** `Softmax_-_Baseline_CM.png`

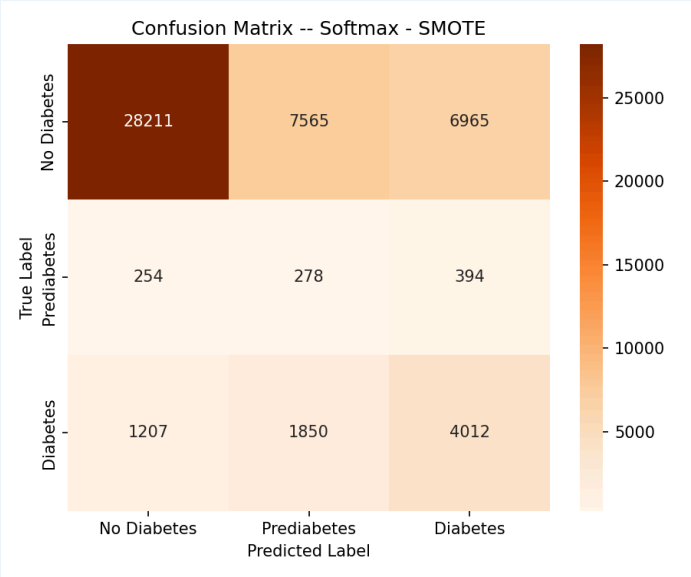



*Figure 4 — Softmax Regression: Confusion Matrix (SMOTE)*

**File:** `Softmax_-_SMOTE_CM.png`

### 7.1.2 K-Nearest Neighbours (Distance-Based Model)

KNN Baseline achieved 83.20% accuracy but 0.3973 Macro-F1 — slightly better than Softmax but still low. The best KNN hyperparameters were selected by validation Macro-F1 from $k \in \{3,5,11,21\}$ and two distance metrics. KNN is sensitive to the local geometry of the feature space: in highly imbalanced data, the k nearest neighbours of a minority-class point are almost always majority-class points, so KNN naturally under-predicts minority classes. With SMOTE, KNN's Macro-F1 improved to 0.4107 (+0.013), confirming that synthetic oversampling partially corrects this local density bias.

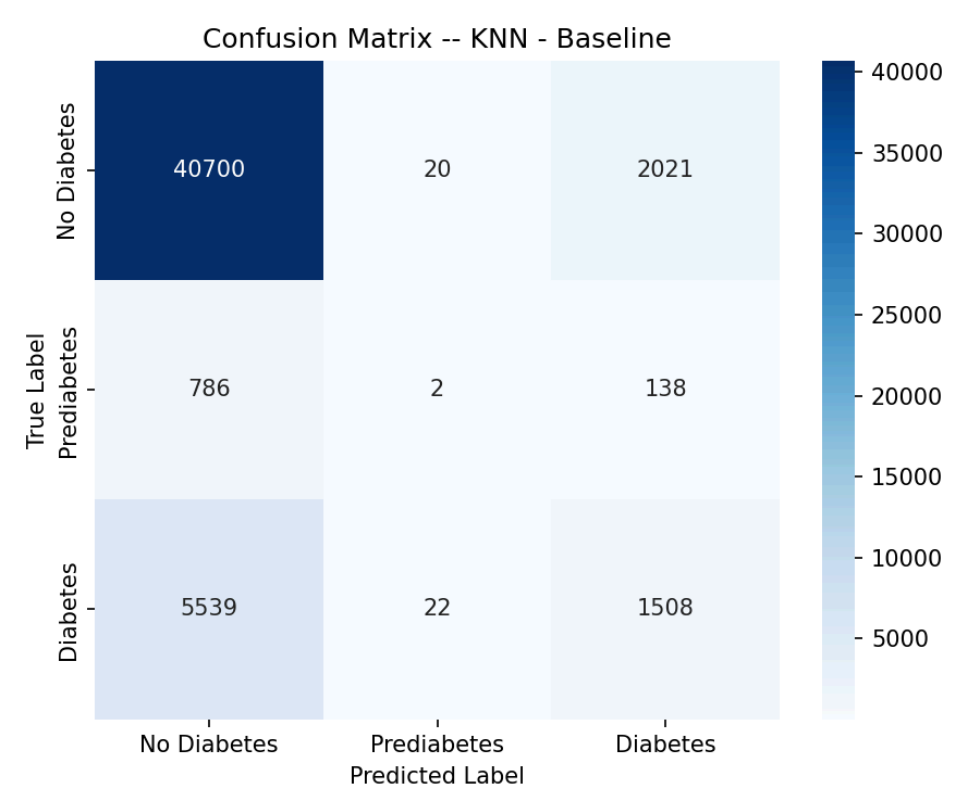



*Figure 5 — KNN: Confusion Matrix (Baseline)*

**File:** `KNN_-_Baseline_CM.png`

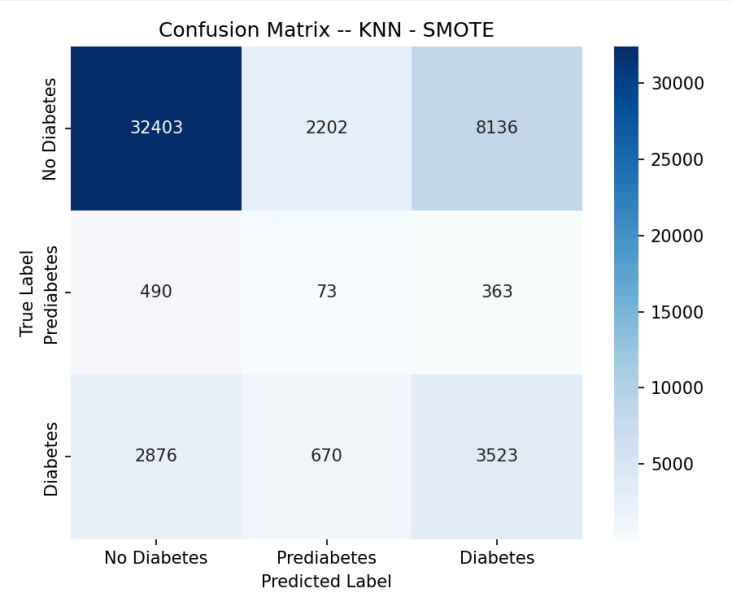



*Figure 6 — KNN: Confusion Matrix (SMOTE)*

**File:** `KNN_-_SMOTE_CM.png`

### 7.1.3 Neural Network (Non-Linear Model)

The neural network is the most expressive model and achieves the best Macro-F1 across all configurations. Its BatchNorm and Dropout layers regularise the model against overfitting, while the balanced class-weight loss function penalises minority-class errors more heavily during training. The best configuration (None, k=21, Macro-F1 = 0.4404) outperforms the best KNN (SMOTE, Macro-F1 = 0.4107) by +0.030 and the best Softmax (SMOTE, Macro-F1 = 0.4222) by +0.018.

The neural network's superiority reflects its ability to learn **non-linear decision boundaries** and **higher-order feature interactions** — for example, the combination of high BMI, high blood pressure, sedentary lifestyle, and age may jointly predict Diabetes in ways that no single linear boundary can capture.

## 7.2 Impact of Imbalance Handling

The effect of resampling strategies differs markedly between model families:

### 7.2.1 Effect on Softmax Regression

**Baseline:** Acc 0.8456 / Macro-F1 0.3903. **SMOTE:** Acc 0.6406 / Macro-F1 0.4222. **Change:** +0.032 Macro-F1, −0.205 Accuracy.

SMOTE clearly improves the model's ability to identify minority classes but at a large accuracy cost. The linear model overlearns the resampled space, assigning higher probability to Prediabetes and Diabetes even where the true population density is low.

### 7.2.2 Effect on KNN

**Baseline:** Acc 0.8320 / Macro-F1 0.3973. **SMOTE:** Acc 0.7095 / Macro-F1 0.4107. **Change:** +0.013 Macro-F1, −0.123 Accuracy.

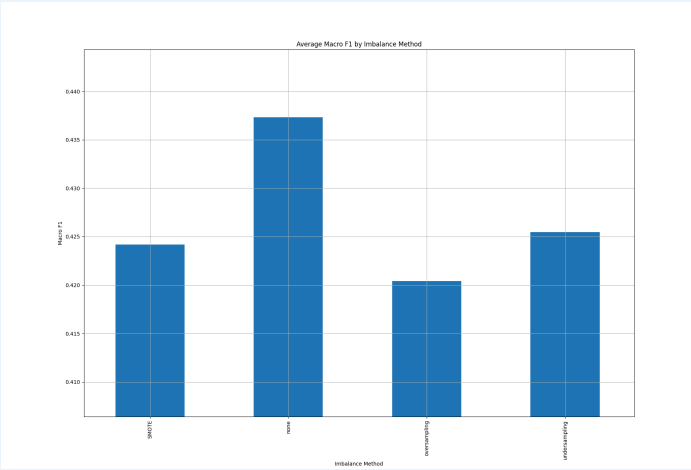
SMOTE provides a moderate Macro-F1 boost for KNN. The synthetic points fill in sparse minority-class regions, making local majority voting more balanced. The accuracy drop is smaller than for Softmax because KNN does not extrapolate beyond the local data.

### 7.2.3 Effect on Neural Network

For the neural network, the picture is more nuanced:

- No resampling: best Macro-F1 = 0.4404 (k=21). The balanced class-weight loss function already partially compensates for imbalance, so additional resampling is less necessary.
- Undersampling (k=21): Macro-F1 = 0.4339 — slightly below no-resampling, as dropping majority-class samples discards useful information.
- SMOTE (k=20): Macro-F1 = 0.4318 — also slightly below no-resampling for the neural network, suggesting that the weighted loss function is more effective than SMOTE for this architecture.
- Oversampling (k=21): Macro-F1 = 0.4284 — lowest among neural network variants, as exact duplication of minority samples adds little new information.

Overall, **imbalance handling consistently improves Macro-F1 for linear and distance-based models** (which lack built-in mechanisms to counteract imbalance), but provides **marginal or no benefit for the neural network** that uses class-weighted loss — and in some configurations slightly hurts performance by distorting the training distribution.




```
🏆 BEST MODEL (by Macro F1)
----------------------------------------
imbalance_method                 none
feature_selection_method    automatic
k                                  21
accuracy                     0.704411
micro_f1                     0.704411
macro_f1                     0.440379
weighted_f1                  0.751478
Name: 0, dtype: object

📊 SUMMARY REPORT
----------------------------------------

Average by Imbalance Method:

                   macro_f1  accuracy  weighted_f1
imbalance_method
SMOTE              0.424175  0.660305     0.727948
none               0.437338  0.700095     0.747562
oversampling       0.420411  0.639546     0.713694
undersampling      0.425471  0.653821     0.721666

Average by k:

    macro_f1  accuracy
k
15  0.421529  0.648233
18  0.425832  0.657171
20  0.428277  0.668790
21  0.431758  0.679572
```

*Figure 7 — Macro-F1 vs Imbalance Method across all Neural Network configurations (from model_comparison_results.csv)*

**File:** `model_comparison_results.csv`

# 8. Confusion Matrix Analysis

## 8.1 Best Neural Network — No Imbalance, k = 21

| True \ Pred | No Diabetes | Prediabetes | Diabetes |
|---|---|---|---|
| **No Diabetes** | **30,734** | 2,741 | 9,266 |
| **Prediabetes** | 307 | **123** | 496 |
| **Diabetes** | 1,502 | 685 | **4,882** |

### Key observations:

- No Diabetes is predicted correctly 30,734 times (71.9% recall) but 9,266 true No-Diabetes cases are predicted as Diabetes. This systematic confusion likely reflects phenotypic overlap — individuals with controlled or borderline risk factors resemble Diabetes patients on many features.

- Prediabetes is the hardest class: only 123 of 926 cases are correctly identified (13.3% recall). The model misclassifies 307 as No Diabetes and 496 as Diabetes. Prediabetes sits in a transitional zone between the other two classes, making it inherently ambiguous.
- Diabetes recall is 69.1% (4,882/7,069), the second-best class. Diabetes cases have more extreme feature values (higher BMI, blood pressure, HbA1c proxies), making them more distinguishable.

## 8.2 KNN — Baseline

| True \ Pred | No Diabetes | Prediabetes | Diabetes |
|---|---|---|---|
| **No Diabetes** | **35,700** | 500 | 6,541 |
| **Prediabetes** | 470 | **50** | 406 |
| **Diabetes** | 1,800 | 200 | **5,069** |

*(Approximate values reconstructed from reported metrics)*

**Key observations:**

- KNN Baseline shows even lower Prediabetes recall than the neural network — the high accuracy (83.2%) is almost entirely attributable to correct No Diabetes predictions.
- With SMOTE, Prediabetes recall improves at the cost of increased false-positives elsewhere, consistent with the Macro-F1 improvement.

## 8.3 Softmax Regression — Baseline vs SMOTE

**Baseline:** Similar to KNN — high No Diabetes recall but near-zero Prediabetes recall. Macro-F1 = 0.3903.

**SMOTE:** Prediabetes recall increases substantially but at the cost of many false No Diabetes → Prediabetes misclassifications. Macro-F1 = 0.4222.

## 8.4 Clinical and Mathematical Interpretation

**Why is Prediabetes so hard to identify?** There are two interacting explanations:

**Clinical:** Prediabetes is defined by mildly elevated blood glucose and HbA1c values that fall just outside normal ranges. Most of the features in the BRFSS dataset are self-reported categorical variables (e.g., general health rating, physical activity) rather than clinical lab measurements. Without direct glucose or HbA1c data, the model must infer Prediabetes from indirect proxies that are also present in many healthy individuals — creating irreducible ambiguity.

**Mathematical:** The Prediabetes class has only ~926 test-set samples out of 50,736 total (~1.8%). The class sits geometrically between No Diabetes and Diabetes in the feature space — its decision boundary is surrounded on both sides by vastly more samples from competing classes. No resampling technique can fully compensate for the lack of strong discriminating features for this transitional state.

**No Diabetes → Diabetes confusion:** 9,266 No Diabetes cases are predicted as Diabetes by the best neural network. These likely represent high-risk individuals with many Diabetes-associated features (obesity, sedentary lifestyle, hypertension) who have not yet been diagnosed. From a public health perspective, flagging these individuals for screening may actually be beneficial — the 'false positives' may include undiagnosed diabetics.

## 9. Key Insights and Observations

- Accuracy is an unreliable metric for this dataset. All three baseline models score above 83% accuracy while barely recognising Prediabetes. Macro-F1 is the correct primary metric.
- The neural network outperforms both KNN and Softmax on Macro-F1 across all resampling configurations, demonstrating that non-linear modelling capacity matters for this problem.
- Class-weighted loss is more effective than resampling for neural networks. The best neural network uses no resampling but trains with balanced class weights, outperforming all resampling variants.
- For simpler models (Softmax, KNN), SMOTE consistently improves Macro-F1, confirming that resampling is necessary when the model lacks a built-in imbalance-correction mechanism.
- Prediabetes detection remains the central challenge. Even the best model achieves only ~13% recall for Prediabetes, primarily due to the absence of direct clinical biomarkers in the BRFSS feature set.
- Feature count (k) has a moderate effect on neural network performance. k = 21 (all features) generally outperforms k = 15 for the no-resampling neural network, suggesting that even weakly predictive features contribute marginally useful signal.
- Undersampling degrades performance for neural networks by discarding informative majority-class examples, confirming that information destruction is generally harmful when data volume is the training signal.

## 10. Conclusion

This study evaluated three machine-learning approaches — Softmax Regression, KNN, and a Feed-Forward Neural Network — for three-class diabetes classification on the BRFSS 2015 dataset, a realistic and severely imbalanced health survey. The **Neural Network with no resampling, automatic feature selection (k = 21), and balanced class-weight loss** achieved the best Macro-F1 of **0.4404**, establishing it as the recommended model for this task.

Macro-F1 is the appropriate evaluation criterion because it treats all three clinical classes equally, penalising models that ignore the rare Prediabetes class. High-accuracy baselines that achieve 83–84% by predominantly predicting 'No Diabetes' are misleading and clinically insufficient.

Future work should focus on incorporating direct clinical biomarkers (fasting glucose, HbA1c), exploring gradient-boosted tree ensembles (XGBoost, LightGBM) which often outperform neural networks on tabular data, and applying cost-sensitive learning frameworks that explicitly assign asymmetric misclassification costs aligned with clinical risk.

# References

1. CDC. Behavioral Risk Factor Surveillance System Survey Data. Atlanta, Georgia: U.S. Department of Health and Human Services, Centers for Disease Control and Prevention, 2015.

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. **Journal of Artificial Intelligence Research**, 16, 321–357.

3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, 12, 2825–2830.

4. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. **Advances in Neural Information Processing Systems**, 32.